



VIT[®]

BHOPAL

Fall semester 2025-26

Vityarthi project

Name: Krishnna Arora

Registration no: 25BAC10022

Class Code :CSE1021

INTRODUCTION

- The Smart Credit Card Management System is a Python-based console application designed to manage basic credit card functionalities. The objective of the project is to simulate real banking operations such as card creation, login authentication, purchases, payments, interest calculation, reward points generation, card blocking, and monitoring credit score. This project allows students to apply programming concepts such as file handling, dictionaries, loops, functions, and modular logic. The system is simple but models core banking functions effectively.

PROBLEM STATEMENT

- Managing credit card activities manually is time-consuming, error-prone, and lacks transparency. Users need a simple system to keep track of their credit card balance, spending, rewards, payments, and card status . The goal is to build an application that allows users to:
Create and manage credit card accounts
Track balances and expenditures
Automatically apply interest
Use reward points
Block/unblock cards
Manage credit score

FUNCTIONAL REQUIREMENTS

- Create Card– allows entering card details and stores in file
- Login– Authenticates using card number and PIN
- Purchase -- Deducts amount, checks limit, adds rewards
- Payment -- Adds rewards reduces balance and increases credit score
- Interest -- Adds 3% interest on outstanding balance
- Card Blocking -- Blocks card to prevent further transaction
- Card Status -- Shows limit, balance, rewards, score, block status

Non-Functional Requirements

- 1. Usability --The interface is text-based and simple for any user to understand
- 2. Reliability --All card details are stored persistently in a text file (cards.txt).
- 3. Performance -- All operations execute within a fraction of a second due to lightweight processing.
- 4. Maintainability -- The code uses multiple small functions making updates easy

Architecture Description

- The system reads card data from a file at startup.
- User selects an operation from the main menu.
- If login is successful, the user accesses card operations .
- All updates are written back to the file immediately.

Design Decisions & Rationale

- Text file storage chosen for simplicity and ease of use.
- Dictionary-based data structure helps in quick lookup.
- Modular functions make the code clean and maintainable.
- Sequential menu-driven flow simplifies navigation.

Implementation Details

- ❑ Programming Language: Python

- ❑ Key Concepts Used:

- ❑ File handling

- ❑ Dictionaries

- ❑ Menu-driven program

- ❑ Functions

- ❑ Error handling

- ❑ String parsing

- ❑ Conditional logic

OUTPUT OF THE CODE OF THE CREDIT CARD
MANAGEMENT SYSTEM IN PYTHON

Card_Mang.py X

Card_Mang.py > ...
def login(cards):

card_menu(num, cards[num], cards)

```
def main():  
    cards= load_cards()  
    while True:  
        print("\n1.Create Card\n2.Login\n3.Exit")  
        x=input("Choose: ")  
        if x=="1":  
            create_card(cards)  
        elif x=="2":  
            login(cards)  
        elif x=="3":  
            break  
        else:  
            print("Invalid")
```

main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\ProjectVIT> python -u "d:\ProjectVIT\Card_Mang.py"

1.Create Card
2.Login
3.Exit
Choose: 1
Card number: 1234567887654321
Name: krishna
PIN: 1234
Limit: 10000

Card_Mang.py X

Card_Mang.py > ...
def login(cards):

card_menu(num, cards[num], cards)

```
def main():  
    cards= load_cards()  
    while True:  
        print("\n1.Create Card\n2.Login\n3.Exit")  
        x=input("Choose: ")  
        if x=="1":  
            create_card(cards)  
        elif x=="2":  
            login(cards)  
        elif x=="3":  
            break  
        else:  
            print("Invalid")
```

main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PIN: 1234
Limit: 10000
Card created

1.Create Card
2.Login
3.Exit
Choose: 2
Card number: 1234567887654321
PIN: 1234
Welcome krishna

1.Purchase
2.Payment
3.Details
4.Block
5.Unblock
6.Interest
7.Logout
Choose:

Card_Mang.py X

Card_Mang.py > ...

def login(cards):

card_menu(num, cards[num], cards)

def main():

cards= load_cards()

while True:

print("\n1.Create Card\n2.Login\n3.Exit")

x=input("Choose: ")

if x=="1":

create_card(cards)

elif x=="2":

login(cards)

elif x=="3":

break

else:

print("Invalid")

main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

1.Purchase
2.Payment
3.Details
4.Block
5.Unblock
6.Interest
7.Logout
Choose: 1
Purchase amount: 1500
Purchase successful

1.Purchase
2.Payment
3.Details
4.Block
5.Unblock
6.Interest
7.Logout
Choose:

Card_Mang.py X

Card_Mang.py > ...

def login(cards):

card_menu(num, cards[num], cards)

def main():

cards= load_cards()

while True:

print("\n1.Create Card\n2.Login\n3.Exit")

x=input("Choose: ")

if x=="1":

create_card(cards)

elif x=="2":

login(cards)

elif x=="3":

break

else:

print("Invalid")

main()

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

5.Unblock
6.Interest
7.Logout
Choose: 3
Card Number: 1234567887654321
Name: krishna
Limit: 10000.0
Balance: 0.0
Rewards: 30.0
Score: 752
Blocked: No

1.Purchase
2.Payment
3.Details
4.Block
5.Unblock
6.Interest
7.Logout
Choose:

Card_Mang.py X

Card_Mang.py > ...

```
def login(cards):  
    card_menu(num, cards[num], cards)  
  
def main():  
    cards= load_cards()  
    while True:  
        print("\n1.Create Card\n2.Login\n3.Exit")  
        x=input("Choose: ")  
        if x=="1":  
            create_card(cards)  
        elif x=="2":  
            login(cards)  
        elif x=="3":  
            break  
        else:  
            print("Invalid")  
  
main()
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

Name: krishna
Limit: 10000.0
Balance: 0.0
Rewards: 30.0
Score: 752
Blocked: No

1.Purchase
2.Payment
3.Details
4.Block
5.Unblock
6.Interest
7.Logout
Choose: 7

1.Create Card
2.Login
3.Exit
Choose: